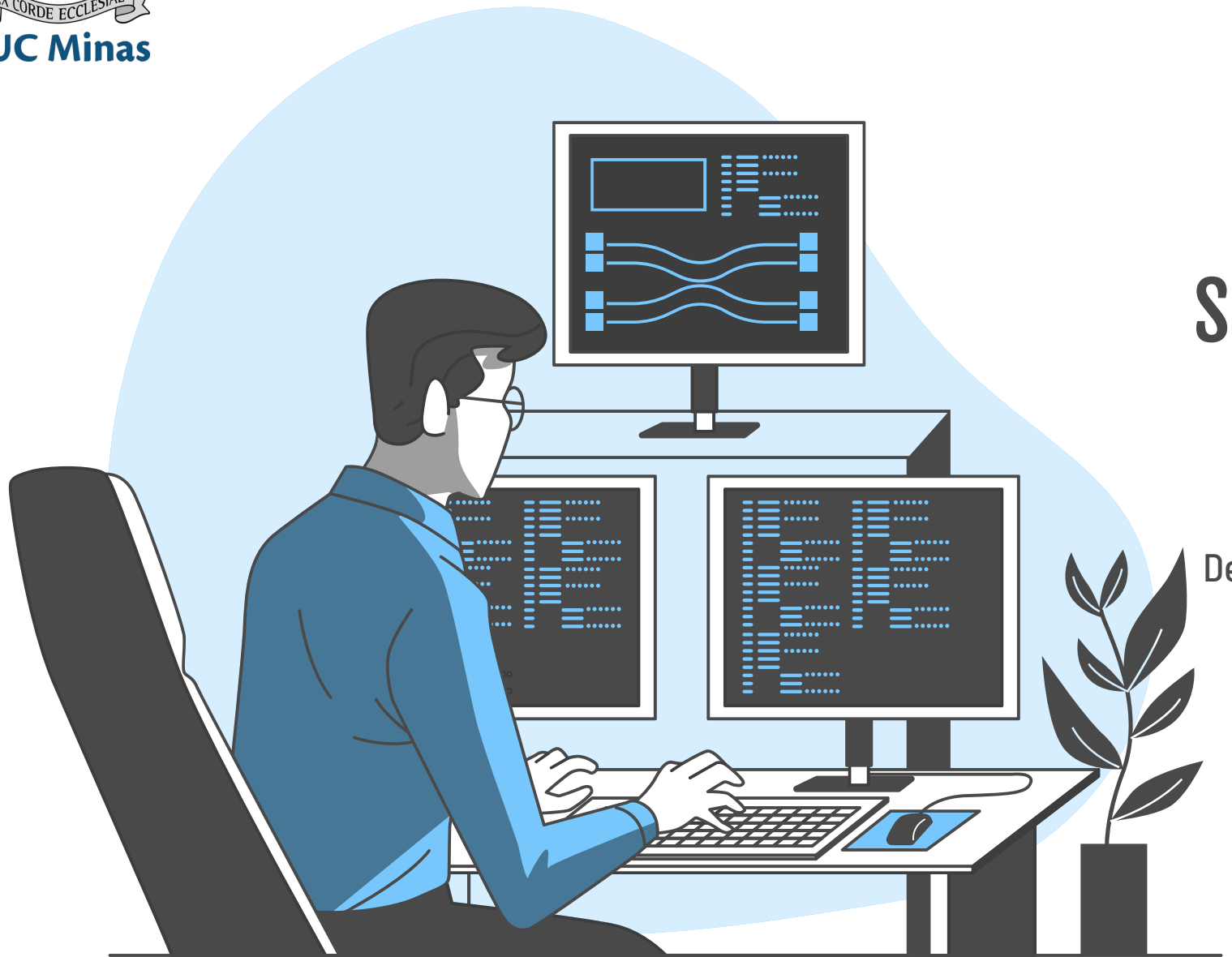
PUC Minas



Aula 09 · Back-end

Spring Security e a classe SecurityConfig

Desenvolvimento e Integração de Aplicações Web

Prof. Dr. João Paulo Aramuni

Engenharia de Software · PUC Minas

github.com/joaopauloaramuni/desenvolvimento-e-integracao-de-aplicacoes-web

Apresentação da aula

Nesta aula você vai

1. Entender **autenticação** x **autorização**
2. Saber o que acontece quando o starter de segurança entra no projeto
3. Escrever um `SecurityFilterChain` com rotas públicas e protegidas
4. Guardar senhas com **BCrypt** e carregar usuários com `UserDetailsService`
5. Decidir quando desligar o **CSRF** e como liberar o **CORS**

Antes de começar

- Aula 04: camadas
- Aula 06 ou 07: banco configurado

Duração sugerida

100 min

Código da disciplina

Projetos de referência na pasta PROJETOS do GitHub

Roteiro

01 Autenticação e autorização

02 O que muda ao adicionar o starter

03 A classe SecurityConfig

04 Usuários e senhas

05 CSRF: ligar ou desligar?

06 CORS com Spring Security

07 Aprofundando: como o login funciona por dentro

01

Autenticação e autorização

Autenticação e autorização

- **Autenticação**: *quem é você?* (login e senha, token JWT).
- **Autorização**: *o que você pode fazer?* (um `USER` lê, só um `ADMIN` apaga).

Erros correspondentes: **401 Unauthorized** = não autenticado; **403 Forbidden** = autenticado, mas sem permissão.

01

Autenticação e autorização

Aula 09 · Back-end

02

O que muda ao adicionar o starter

O que muda ao adicionar o starter

pom.xml

```xml
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>
```

O que muda ao adicionar o starter

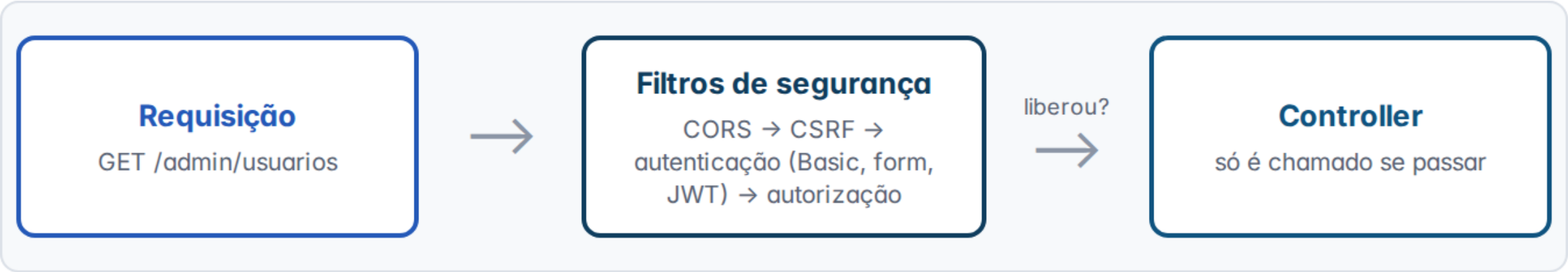
Terminal

```
aluno@pc:~/Back$ ./mvnw spring-boot:run
Using generated security password: 3f9a1c2e-8b7d-4e6f-a5c4-2b1d0e9f8a7c
This generated password is for development use only. Your security configuration must be updated before
running your application in production.
Tomcat started on port 8080 (http)
```

Sem nenhuma configuração, **todas** as rotas passam a exigir login. O usuário é `user` e a senha aparece no console.

O que muda ao adicionar o starter

O Spring Security funciona como uma **cadeia de filtros** que roda **antes** do controller. Cada requisição passa pelos filtros; se algum barrar, o controller nem é chamado.

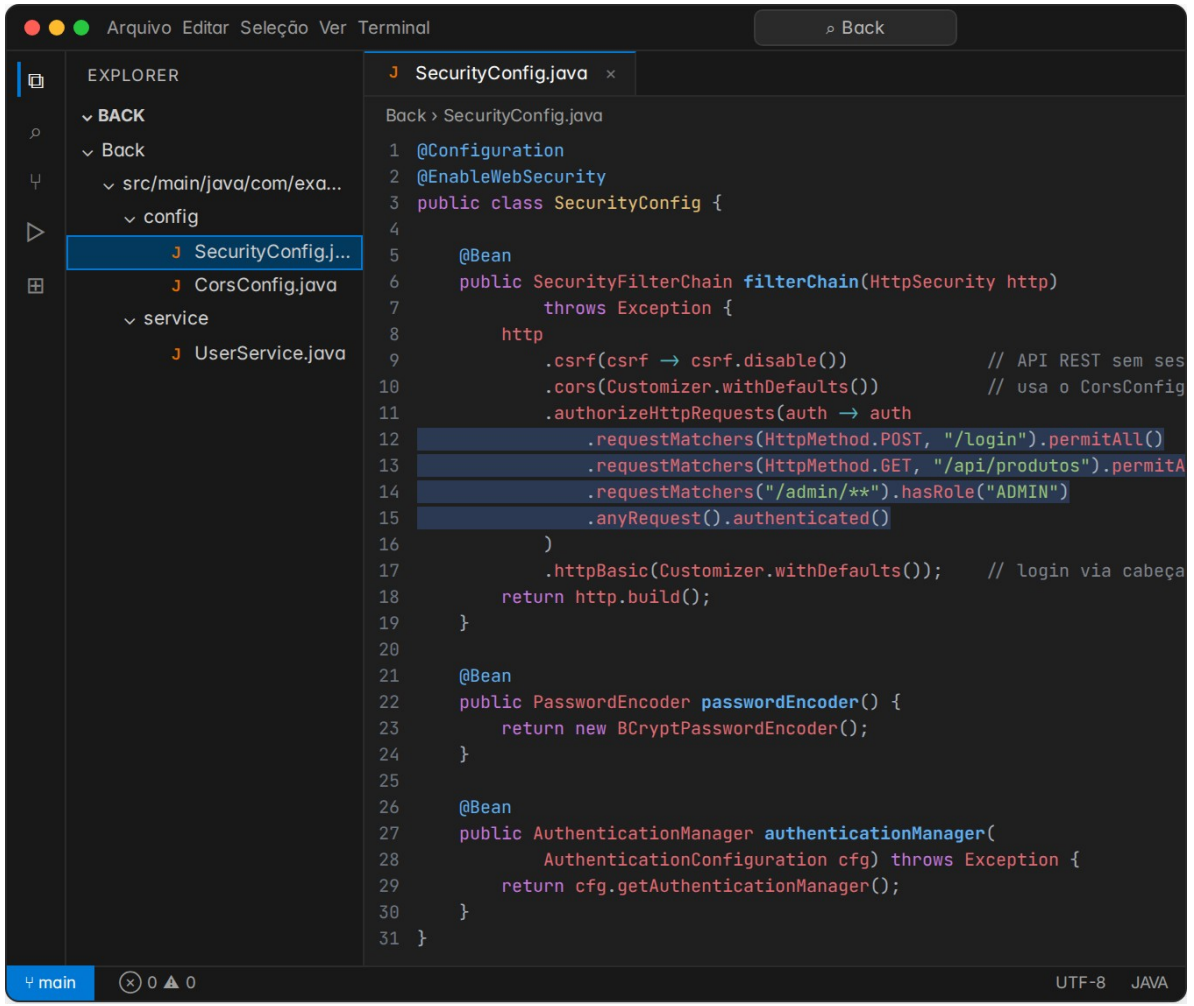


A `SecurityFilterChain` decide se a requisição chega ao controller.

03

A classe SecurityConfig

A classe SecurityConfig



Fica no pacote `config`. É nela que você diz **quais rotas são públicas**, **quais papéis acessam o quê** e **como** o usuário se autentica.

`SecurityConfig` baseada no `Exemplo_Arquitetura_Completa`: regras de acesso, BCrypt e o `AuthenticationManager`.

Lendo as regras

As regras são avaliadas **de cima para baixo**; a primeira que casar vence. Por isso `anyRequest()` vem sempre por último.

Método	Significado
`permitAll()`	Qualquer um, mesmo sem login
`authenticated()`	Qualquer usuário logado
`hasRole("ADMIN")`	Só quem tem o papel `ROLE_ADMIN` (o prefixo `ROLE_` é automático)
`hasAnyRole("ADMIN","PROF")`	Um dos papéis
`denyAll()`	Ninguém

Formas de autenticar

Configuração	Como o usuário se identifica	Uso típico
`.httpBasic(...)`	Cabeçalho `Authorization: Basic base64(user:senha)` em toda requisição	Testes, APIs internas
`.formLogin(...)`	Página de login, depois cookie de sessão	Projetos **Thymeleaf**
Filtro JWT	Cabeçalho `Authorization: Bearer`	APIs consumidas por **React** (aula 10)

04

Usuários e senhas

Nunca guarde a senha em texto puro

O `BCryptPasswordEncoder` gera um *hash* com *salt* aleatório. A mesma senha gera textos diferentes a cada vez, e não é possível voltar ao original. Para conferir, o Spring usa `encoder.matches(senhaDigitada, hashDoBanco)`.

```
Senha e seu hash BCrypt

1234  →  $2a$10$N9qo8uLOickgx2ZMRZoMyeIjZAgcfl7p92ldGxad68LJZdL17lhWy
```

UserDetailsService: de onde vêm os usuários

O Spring precisa de um bean que, dado um *username*, devolva o usuário com senha e papéis. Os projetos mostram duas versões:

Versão com banco

```java
// Usuários do banco (Supabase/Mongo)
@Bean
public UserDetailsService uds(UserService s) {
  return username -> s.findByUsername(username)
    .map(u -> User.builder()
        .username(u.getUsername())
        .password(u.getPassword()) // hash
        .roles(u.getRole().name())
        .build())
    .orElseThrow(() ->
        new UsernameNotFoundException(username));
}
```

Versão em memória

```java
// Usuários em memória (lidos do
// application.properties)
@Bean
public UserDetailsService uds(
        PasswordEncoder enc) {
  return new InMemoryUserDetailsManager(
    User.withUsername("admin")
        .password(enc.encode("1234"))
        .roles("ADMIN").build(),
    User.withUsername("joao")
        .password(enc.encode("4321"))
        .roles("USER").build());
}
```

UserDetailsService: de onde vêm os usuários

No `Exemplo_Arquitetura_Completa`, os usuários iniciais ficam no `application.properties` (`app.security.users[0].username=...`), são lidos por `SecurityProperties` e gravados no banco com a senha já codificada por um `CommandLineRunner` na inicialização.

04

Usuários e senhas

Aula 09 · Back-end

05

CSRF: ligar ou desligar?

CSRF: ligar ou desligar?

CSRF (*Cross-Site Request Forgery*) é um ataque em que outro site faz o navegador da vítima enviar um formulário usando o **cookie de sessão** dela. A proteção exige um token secreto em cada `POST`.

- **Projeto Thymeleaf com `formLogin`** (sessão por cookie): **mantenha ligado**. O Thymeleaf já insere o token nos formulários.
- **API REST com JWT** (sem cookie de sessão): pode **desligar**, porque o token vai no cabeçalho, que outro site não consegue forjar.

05

CSRF: ligar ou desligar?

Aula 09 · Back-end

06

CORS com Spring Security

CORS com Spring Security

Quando o React (porta 5173 ou Vercel) chama a API, o navegador manda antes uma requisição `OPTIONS` (*preflight*). Com Security no projeto, ela precisa passar pelos filtros. Ligue `.cors(...)` e forneça um `CorsConfigurationSource`:

config/SecurityConfig.java (CORS)

```java
@Bean
public CorsConfigurationSource corsConfigurationSource(CorsProperties props) {
    var cfg = new CorsConfiguration();
    cfg.setAllowedOriginPatterns(props.getAllowedOrigins());   // localhost:5173, *.vercel.app
    cfg.setAllowedMethods(List.of("GET", "POST", "PUT", "DELETE", "OPTIONS"));
    cfg.setAllowedHeaders(List.of("*"));                       // inclui Authorization
    var source = new UrlBasedCorsConfigurationSource();
    source.registerCorsConfiguration("/**", cfg);
    return source;
}
```

CORS com Spring Security

Erro comum

Login funciona no cliente HTTP, mas o React recebe **erro de CORS** ou **401 no OPTIONS**: falta `.cors(...)` no `SecurityFilterChain`. O `WebMvcConfigurer` com `addCorsMappings` sozinho não basta quando o Spring Security está no projeto, porque os filtros de segurança rodam antes do MVC.

07

Aprofundando: como o login funciona por dentro

Aprofundando: como o login funciona por dentro

Peça	Responsabilidade
`SecurityFilterChain`	Lista ordenada de filtros que toda requisição atravessa
`AuthenticationManager`	Recebe usuário e senha e decide se são válidos
`DaoAuthenticationProvider`	Implementação padrão: busca o usuário e compara a senha
`UserDetailsService`	Carrega o usuário pelo username (banco ou memória)
`PasswordEncoder`	Compara a senha digitada com o hash BCrypt
`SecurityContextHolder`	Guarda quem está autenticado durante a requisição

Segurança por método

```
@PreAuthorize
```

```java
@Configuration
@EnableMethodSecurity            // liga @PreAuthorize
public class SecurityConfig { ... }

@Service
public class UsuarioService {

    @PreAuthorize("hasRole('ADMIN')")
    public void excluir(Long id) { ... }

    @PreAuthorize("#username == authentication.name or hasRole('ADMIN')")
    public Usuario perfil(String username) { ... }   // cada um vê o seu
}
```

Role x authority

Role (papel)

- `hasRole("ADMIN")`
- Procura a authority `ROLE_ADMIN`
- Grupos de usuários: ADMIN, USER, PROFESSOR

vs

Authority (permissão)

- `hasAuthority("alunos:excluir")`
- Comparação exata do texto
- Permissões finas por ação

Login com formulário (projetos Thymeleaf)

SecurityConfig com formLogin

```
http
    .authorizeHttpRequests(auth -> auth
        .requestMatchers("/login", "/register", "/css/**", "/images/**").permitAll()
        .requestMatchers("/admin/**").hasRole("ADMIN")
        .anyRequest().authenticated())
    .formLogin(form -> form
        .loginPage("/login")                // seu template login.html
        .defaultSuccessUrl("/home", true)
        .failureUrl("/login?erro"))
    .logout(out -> out
        .logoutSuccessUrl("/login?saiu")
        .permitAll());
```

Testando a segurança

```
@WithMockUser
```

```
@WebMvcTest(AdminController.class)
@Import(SecurityConfig.class)
class AdminControllerTest {
    @Autowired MockMvc mvc;

    @Test
    void anonimoRecebe401() throws Exception {
        mvc.perform(get("/admin/usuarios")).andExpect(status().isUnauthorized());
    }

    @Test
    @WithMockUser(roles = "USER")
    void usuarioComumRecebe403() throws Exception {
        mvc.perform(get("/admin/usuarios")).andExpect(status().isForbidden());
    }

    @Test
    @WithMockUser(roles = "ADMIN")
    void adminRecebe200() throws Exception {
        mvc.perform(get("/admin/usuarios")).andExpect(status().isOk());
    }
}
```

Checklist de segurança da API

Senhas

Sempre BCrypt; nunca devolva o hash em nenhuma resposta.

Mensagens de erro

"Usuário ou senha inválidos": não diga qual dos dois errou.

HTTPS

Render e Vercel já entregam HTTPS; nunca use `http://` em produção.

Segredos

Chaves e senhas só em variáveis de ambiente.

CORS

Liste as origens; nada de `*` com credenciais.

Força bruta

reCAPTCHA no login (projeto `Captcha`) ou limite de tentativas.

Aprofundando: como o login funciona por dentro

Erro comum

Esquecer `.requestMatchers("/error").permitAll()` em alguns cenários faz o Spring Security responder 401 até para páginas de erro, escondendo a exceção real. Se toda requisição volta 401 sem motivo aparente, libere `/error` e veja a mensagem verdadeira.

08

Quiz de revisão

Quiz · questão 1 de 5

Pergunta

Qual status indica “autenticado, mas sem permissão”?

A 401

B 403

C 404

D 500

Resposta

Qual status indica “autenticado, mas sem permissão”?

A 401

✓ 403

C 404

D 500

Por quê: 401 é não autenticado.

Quiz · questão 2 de 5

Pergunta

Em `hasRole("ADMIN")`, qual authority o Spring procura?

A ADMIN

B ROLE_ADMIN

C admin

D AUTH_ADMIN

Quiz · questão 2 de 5

Resposta

Em `hasRole("ADMIN")`, qual authority o Spring procura?

A ADMIN

✓ ROLE_ADMIN

C admin

D AUTH_ADMIN

Por quê: O prefixo ROLE_ é adicionado automaticamente.

Quiz · questão 3 de 5

Pergunta

Por que as regras de `authorizeHttpRequests` precisam de ordem?

A. Por estética

B. A primeira regra que casar vence

C. O Spring ordena sozinho

D. Só a última vale

Quiz · questão 3 de 5

Resposta

Por que as regras de `authorizeHttpRequests` precisam de ordem?

A. Por estética

✓ A primeira regra que casar vence

C. O Spring ordena sozinho

D. Só a última vale

Por quê: Por isso `anyRequest()` vem sempre no final.

Pergunta

Quando é razoável desligar o CSRF?

A. Em projetos Thymeleaf com sessão

B. Em APIs REST stateless com JWT no cabeçalho

C. Sempre

D. Nunca

Resposta

Quando é razoável desligar o CSRF?

A Em projetos Thymeleaf com sessão

✓ Em APIs REST stateless com JWT no cabeçalho

C Sempre

D Nunca

Por quê: Sem cookie de sessão, outro site não consegue forjar a credencial.

Pergunta

Com Spring Security, o que falta quando o React recebe 401 no OPTIONS?

A Um novo controller

B `.cors(...)` na SecurityFilterChain

C Trocar a porta

D Desligar o BCrypt

Resposta

Com Spring Security, o que falta quando o React recebe 401 no OPTIONS?

A. Um novo controller

✓ `.cors(...)` na SecurityFilterChain

C. Trocar a porta

D. Desligar o BCrypt

Por quê: Os filtros de segurança rodam antes do Spring MVC.

Fechamento

Resumo da aula

- ✓ **Autenticação** = quem é; **autorização** = o que pode. 401 x 403.
- ✓ Com o starter, tudo fica protegido; a `SecurityFilterChain` na `SecurityConfig` define as regras.
- ✓ Regras de cima para baixo: `permitAll`, `hasRole`, e `anyRequest().authenticated()` no fim.
- ✓ Senhas sempre com **BCrypt**; usuários vêm de um `UserDetailsService` (banco ou memória).
- ✓ CSRF ligado com sessão/Thymeleaf; desligado em API REST com JWT. CORS precisa de `.cors(...)`.

Exercícios

1. Adicione Spring Security ao CRUD de alunos: `GET` público, `POST/PUT/DELETE` só autenticado.

2. Crie dois usuários em memória (USER e ADMIN) e proteja `DELETE` com `hasRole("ADMIN")`.

3. Teste no cliente HTTP com Basic Auth e confirme os códigos 401 e 403.

4. Grave os usuários no banco com BCrypt e confira no Supabase/Atlas que a senha não aparece em texto.

Para ir além

- docs.spring.io/spring-security/reference/servlet/architecture.html
- Projetos `SecureLogin`, `SecureLoginPUC` e `Exemplo_Arquitetura_Completa`

Repositório da disciplina

Todos os projetos citados estão na pasta PROJETOS

Abrir no GitHub

Links e versões citados podem ter mudado. Confira sempre a documentação oficial.



Obrigado!

Dúvidas sobre a aula 09?

Prof. Dr. João Paulo Aramuni
Engenharia de Software · PUC Minas
Desenvolvimento e Integração de Aplicações Web

GitHub da disciplina
https://github.com/joaopauloaramuni/desenvolvimento-e-integracao-de-aplicacoes-web